nen-mcp-server全コード集.md

これは毎回目的もなく作ってるやつだよ。 今回は、Gemini CLIがPowerShellのコマンドで文書読むときに日本語の文字化けがひどすぎてムカついたから作ったよ。多分英語圏以外の人も使えるはずだよ。

コード

処理系

src/main.rs

```
mod mcp;
mod encoding;
mod tools;
mod parser;

use std::io::{self, BufRead, Write};
use clap::{Parser, Subcommand};
use mcp::{JsonRpcRequest, JsonRpcResponse, InitializeResult, ServerCapabilities,
ServerInfo, ToolCapabilities, ToolListResult, Tool, CallToolRequest,
CallToolResult, ToolContent};
use serde_json::{to_string, Value};
use tools::safe_read::safe_read;
use tools::get_outline::get_outline;
use tools::inspect_file::inspect_file;
use tools::read_hex::read_hex;

#[derive(Parser)]
#[command(author, version, about, long_about = None)]
struct Cli {
    /// Run as an MCP server (default behavior)
    #[arg(short, long)]
    mcp: bool,

    #[command(subcommand)]
    command: Option<Commands>,
}

#[derive(Subcommand)]
enum Commands {
    /// Reads a file with automatic encoding detection.
    SafeRead {
        path: String,
        /// Optional range: start end (e.g. --range 0 100)
        #[arg(short, long, num_args = 2)]
        range: Option<Vec<usize>>,
        /// Read from the end of the file
        #[arg(short, long)]
        tail: Option<usize>,
    },
```

```
    /// Extracts a high-level outline of the file (functions, classes, etc.) using
Tree-sitter.
    GetOutline {
        path: String,
    },
    /// Provides detailed file metadata and optional keyword search with context.
    InspectFile {
        path: String,
        /// Keyword search query
        #[arg(short, long)]
        search: Option<String>,
    },
    /// Reads a file and returns a traditional hex dump format.
    ReadHex {
        path: String,
        /// Starting offset
        #[arg(short, long)]
        offset: Option<u64>,
        /// Number of bytes to read
        #[arg(short, long)]
        length: Option<usize>,
    },
}

#[cfg(windows)]
fn fix_windows_console() {
    use windows_sys::Win32::System::Console::{SetConsoleOutputCP, SetConsoleCP};
    unsafe {
        SetConsoleOutputCP(65001);
        SetConsoleCP(65001);
    };
}

#[cfg(not(windows))]
fn fix_windows_console() {}

fn write_response(res_str: &str) {
    let mut stdout = std::io::stdout();
    stdout.write_all(res_str.as_bytes()).unwrap();
    stdout.write_all(b"\n").unwrap();
    stdout.flush().unwrap();
}

/// Dispatches tool calls to the appropriate implementation.
fn handle_tool_call(name: &str, arguments: Value) -> CallToolResult {
    match name {
        "safe_read" => {
            let path = arguments["path"].as_str().unwrap_or("");
            let range = arguments["range"].as_array().and_then(|a| {
                if a.len() == 2 {
                    let start = a[0].as_u64()? as usize;
                    let end = a[1].as_u64()? as usize;
                    Some([start, end])
                } else {
```

```
                    None
                }
            });
            let tail = arguments["tail"].as_u64().map(|n| n as usize);

            match safe_read(path, range, tail) {
                Ok((content, encoding)) => {
                    let text = format!("[推定エンコーディング: {}]\n{}", encoding,
content);
                    CallToolResult {
                        content: vec![ToolContent::Text { text }],
                        is_error: Some(false),
                    }
                },
                Err(e) => CallToolResult {
                    // {:#} を使うことで、anyhow のコンテキスト（エラーの連鎖）を表示
する
                    content: vec![ToolContent::Text { text: format!("エラーが発生し
ました:\n{:#}", e) }],
                    is_error: Some(true),
                }
            }
        },
        "get_outline" => {
            let path = arguments["path"].as_str().unwrap_or("");
            match get_outline(path) {
                Ok(outline) => CallToolResult {
                    content: vec![ToolContent::Text { text: outline }],
                    is_error: Some(false),
                },
                Err(e) => CallToolResult {
                    content: vec![ToolContent::Text { text: format!("アウトライン取
得中にエラーが発生しました:\n{:#}", e) }],
                    is_error: Some(true),
                }
            }
        },
        "inspect_file" => {
            let path = arguments["path"].as_str().unwrap_or("");
            let search_query = arguments["search_query"].as_str().map(|s|
s.to_string());
            match inspect_file(path, search_query) {
                Ok(result_json) => CallToolResult {
                    content: vec![ToolContent::Text { text: result_json }],
                    is_error: Some(false),
                },
                Err(e) => CallToolResult {
                    content: vec![ToolContent::Text { text: format!("ファイル解析中
にエラーが発生しました:\n{:#}", e) }],
                    is_error: Some(true),
                }
            }
        },
        "read_hex" => {
```

```
            let path = arguments["path"].as_str().unwrap_or("");
            let offset = arguments["offset"].as_u64();
            let length = arguments["length"].as_u64().map(|n| n as usize);
            match read_hex(path, offset, length) {
                Ok(hex_dump) => CallToolResult {
                    content: vec![ToolContent::Text { text: hex_dump }],
                    is_error: Some(false),
                },
                Err(e) => CallToolResult {
                    content: vec![ToolContent::Text { text: format!("バイナリ読み取り中にエラーが発生しました:\n{:#}", e) }],
                    is_error: Some(true),
                }
            }
        },
        _ => CallToolResult {
            content: vec![ToolContent::Text { text: format!("ツール '{}' は見つかりませんでした。", name) }],
            is_error: Some(true),
        }
    }
}

fn run_mcp_server() {
    let stdin = io::stdin();
    for line in stdin.lock().lines() {
        let line = match line {
            Ok(line) => line,
            Err(_) => break,
        };

        if line.trim().is_empty() {
            continue;
        }

        let request: JsonRpcRequest = match serde_json::from_str(&line) {
            Ok(req) => req,
            Err(_) => continue,
        };

        if request.method == "initialize" {
            let result = InitializeResult {
                protocol_version: "2024-11-05".to_string(),
                capabilities: ServerCapabilities {
                    tools: Some(ToolCapabilities {
                        list_changed: Some(false),
                    }),
                },
                server_info: ServerInfo {
                    name: "nen-mcp-server".to_string(),
                    version: env!("CARGO_PKG_VERSION").to_string(),
                },
            };
```

```
            let response = JsonRpcResponse::success(
                request.id,
                serde_json::to_value(result).unwrap(),
            );

            if let Ok(res_str) = to_string(&response) {
                write_response(&res_str);
            }
        } else if request.method == "tools/list" {
            let result = ToolListResult {
                tools: vec![
                    Tool {
                        name: "safe_read".to_string(),
                        description: Some("Reads a file with automatic encoding
detection.".to_string()),
                        input_schema: serde_json::json!({
                            "type": "object",
                            "properties": {
                                "path": { "type": "string" },
                                "range": {
                                    "type": "array",
                                    "items": { "type": "integer" },
                                    "minItems": 2,
                                    "maxItems": 2
                                },
                                "tail": { "type": "integer" }
                            },
                            "required": ["path"]
                        }),
                    },
                    Tool {
                        name: "get_outline".to_string(),
                        description: Some("Extracts a high-level outline of the
file (functions, classes, etc.) using Tree-sitter.".to_string()),
                        input_schema: serde_json::json!({
                            "type": "object",
                            "properties": {
                                "path": { "type": "string" }
                            },
                            "required": ["path"]
                        }),
                    },
                    Tool {
                        name: "inspect_file".to_string(),
                        description: Some("Provides detailed file metadata and
optional keyword search with context.".to_string()),
                        input_schema: serde_json::json!({
                            "type": "object",
                            "properties": {
                                "path": { "type": "string" },
                                "search_query": { "type": "string" }
                            },
                            "required": ["path"]
                        }),
```

```
                    },
                    Tool {
                        name: "read_hex".to_string(),
                        description: Some("Reads a file and returns a traditional
hex dump format.".to_string()),
                        input_schema: serde_json::json!({
                            "type": "object",
                            "properties": {
                                "path": { "type": "string" },
                                "offset": { "type": "integer" },
                                "length": { "type": "integer" }
                            },
                            "required": ["path"]
                        }),
                    }
                ],
            };
            let response = JsonRpcResponse::success(request.id,
serde_json::to_value(result).unwrap());
            if let Ok(res_str) = to_string(&response) {
                write_response(&res_str);
            }
        } else if request.method == "tools/call" || request.method ==
"mcp_tools/call" {
            let call_req: CallToolRequest = match
serde_json::from_value(request.params.clone()) {
                Ok(req) => req,
                Err(_) => {
                    let response = JsonRpcResponse::error(request.id, -32602,
"Invalid params");
                    if let Ok(res_str) = to_string(&response) {
                        write_response(&res_str);
                    }
                    continue;
                }
            };

            let result = handle_tool_call(&call_req.name,
call_req.arguments.unwrap_or(Value::Null));
            let response = JsonRpcResponse::success(request.id,
serde_json::to_value(result).unwrap());
            if let Ok(res_str) = to_string(&response) {
                write_response(&res_str);
            }
        }
    }
}

fn main() {
    fix_windows_console();
    let cli = Cli::parse();

    if cli.mcp || cli.command.is_none() {
        run_mcp_server();
```

```
    } else if let Some(command) = cli.command {
        let result = match command {
            Commands::SafeRead { path, range, tail } => {
                let range_val = range.map(|r| serde_json::json!
(r)).unwrap_or(Value::Null);
                let tail_val = tail.map(|t| serde_json::json!
(t)).unwrap_or(Value::Null);
                let args = serde_json::json!({
                    "path": path,
                    "range": range_val,
                    "tail": tail_val
                });
                handle_tool_call("safe_read", args)
            },
            Commands::GetOutline { path } => {
                let args = serde_json::json!({ "path": path });
                handle_tool_call("get_outline", args)
            },
            Commands::InspectFile { path, search } => {
                let args = serde_json::json!({
                    "path": path,
                    "search_query": search
                });
                handle_tool_call("inspect_file", args)
            },
            Commands::ReadHex { path, offset, length } => {
                let args = serde_json::json!({
                    "path": path,
                    "offset": offset,
                    "length": length
                });
                handle_tool_call("read_hex", args)
            }
        };

        for content in result.content {
            match content {
                ToolContent::Text { text } => {
                    // Use write_all to ensure UTF-8 output without Rust's auto-
conversion
                    let _ = std::io::stdout().write_all(text.as_bytes());
                    let _ = std::io::stdout().write_all(b"\n");
                }
            }
        }
        if result.is_error == Some(true) {
            std::process::exit(1);
        }
    }
}
```

src/encoding.rs

```
use chardetng::EncodingDetector;

pub fn decode_to_utf8(bytes: &[u8]) -> (String, &'static str) {
    let mut detector = EncodingDetector::new();
    detector.feed(bytes, true);
    let encoding = detector.guess(None, true);
    let (decoded, _, _) = encoding.decode(bytes);
    (decoded.into_owned(), encoding.name())
}

#[cfg(test)]
mod tests {
    use super::*;
    #[test]
    fn test_decode_sjis() {
        let sjis_bytes = vec![0x82, 0xB1, 0x82, 0xF1, 0x82, 0xC9, 0x82, 0xBF,
0x82, 0xCD];
        let (decoded, encoding) = decode_to_utf8(&sjis_bytes);
        assert_eq!(decoded, "こんにちは");
        assert!(encoding.to_lowercase().contains("shift"));
    }
}
```

src/mcp.rs

```
use serde::{Deserialize, Serialize};
use serde_json::Value;

#[derive(Debug, Serialize, Deserialize)]
pub struct JsonRpcRequest {
    pub jsonrpc: String,
    pub id: Value,
    pub method: String,
    pub params: Value,
}

#[derive(Debug, Serialize, Deserialize)]
pub struct JsonRpcResponse {
    pub jsonrpc: String,
    pub id: Value,
    #[serde(skip_serializing_if = "Option::is_none")]
    pub result: Option<Value>,
    #[serde(skip_serializing_if = "Option::is_none")]
    pub error: Option<JsonRpcError>,
}

#[derive(Debug, Serialize, Deserialize)]
pub struct JsonRpcError {
    pub code: i32,
    pub message: String,
```

```rust
    #[serde(skip_serializing_if = "Option::is_none")]
    pub data: Option<Value>,
}

#[derive(Debug, Serialize, Deserialize)]
pub struct InitializeResult {
    pub protocol_version: String,
    pub capabilities: ServerCapabilities,
    pub server_info: ServerInfo,
}

#[derive(Debug, Serialize, Deserialize)]
pub struct ServerCapabilities {
    #[serde(skip_serializing_if = "Option::is_none")]
    pub tools: Option<ToolCapabilities>,
}

#[derive(Debug, Serialize, Deserialize)]
pub struct ToolCapabilities {
    #[serde(skip_serializing_if = "Option::is_none")]
    pub list_changed: Option<bool>,
}

#[derive(Debug, Serialize, Deserialize)]
pub struct ServerInfo {
    pub name: String,
    pub version: String,
}

#[derive(Debug, Serialize, Deserialize)]
pub struct ToolListResult {
    pub tools: Vec<Tool>,
}

#[derive(Debug, Serialize, Deserialize)]
pub struct Tool {
    pub name: String,
    pub description: Option<String>,
    pub input_schema: Value,
}

#[derive(Debug, Serialize, Deserialize)]
pub struct CallToolRequest {
    pub name: String,
    pub arguments: Option<Value>,
}

#[derive(Debug, Serialize, Deserialize)]
pub struct CallToolResult {
    pub content: Vec<ToolContent>,
    #[serde(skip_serializing_if = "Option::is_none")]
    pub is_error: Option<bool>,
}
```

```rust
#[derive(Debug, Serialize, Deserialize)]
#[serde(tag = "type")]
pub enum ToolContent {
    #[serde(rename = "text")]
    Text { text: String },
}

impl JsonRpcResponse {
    pub fn success(id: Value, result: Value) -> Self {
        Self {
            jsonrpc: "2.0".to_string(),
            id,
            result: Some(result),
            error: None,
        }
    }

    pub fn error(id: Value, code: i32, message: &str) -> Self {
        Self {
            jsonrpc: "2.0".to_string(),
            id,
            result: None,
            error: Some(JsonRpcError {
                code,
                message: message.to_string(),
                data: None,
            }),
        }
    }
}
```

src/parser/mod.rs

```rust
use tree_sitter::Language;

pub fn get_language(extension: &str) -> Option<Language> {
    match extension.to_lowercase().as_str() {
        "rs" => Some(tree_sitter_rust::language()),
        "py" => Some(tree_sitter_python::language()),
        "cs" => Some(tree_sitter_c_sharp::language()),
        _ => None,
    }
}

#[cfg(test)]
mod tests {
    use super::*;
    #[test]
    fn test_get_language() {
        assert!(get_language("rs").is_some());
        assert!(get_language("py").is_some());
        assert!(get_language("cs").is_some());
```

```
        assert!(get_language("unknown").is_none());
    }
}
```

src/tools/mod.rs

```
pub mod safe_read;
pub mod get_outline;
pub mod inspect_file;
pub mod read_hex;

use std::path::PathBuf;
use anyhow::{Result, Context};

/// Validates that the path exists, is a file, and returns its canonicalized form.
pub fn validate_and_canonicalize(path: &str) -> Result<PathBuf> {
    let path_buf = dunce::canonicalize(path)
        .with_context(|| format!("指定されたパス '{}' の正規化（存在確認）に失敗しました。パスが正しいか、権限があるか確認してください。", path))?;

    if !path_buf.is_file() {
        return Err(anyhow::anyhow!("パス '{}' は存在しますが、ファイルではありません。このツールは単一のファイルのみを対象としています。", path_buf.display()));
    }
    Ok(path_buf)
}
```

src/tools/safe_read.rs

```
use std::fs::File;
use std::io::{Read, Seek, SeekFrom};
use crate::encoding::decode_to_utf8;
use anyhow::{Result, Context};
use super::validate_and_canonicalize;

pub fn safe_read(
    path: &str,
    range: Option<[usize; 2]>,
    tail: Option<usize>,
) -> Result<(String, &'static str)> {
    let path_buf = validate_and_canonicalize(path)?;
    let mut file = File::open(&path_buf)
        .with_context(|| format!("ファイル '{}' を開くことができませんでした。", path))?;
    let metadata = file.metadata()
        .with_context(|| format!("ファイル '{}' のメタデータ（サイズ等）を取得できませんでした。", path))?;
    let file_size = metadata.len() as usize;
```

```
    let mut buffer = Vec::new();

    if let Some(n) = tail {
        let start = if file_size > n { file_size - n } else { 0 };
        file.seek(SeekFrom::Start(start as u64))
            .with_context(|| format!("ファイル '{}' の末尾 {} バイトへのシークに失敗
しました。", path, n))?;
        file.read_to_end(&mut buffer)
            .with_context(|| format!("ファイル '{}' の末尾内容の読み取りに失敗しまし
た。", path))?;
    } else if let Some([start, end]) = range {
        let actual_end = std::cmp::min(end, file_size);
        if start < actual_end {
            let len = actual_end - start;
            file.seek(SeekFrom::Start(start as u64))
                .with_context(|| format!("ファイル '{}' の指定位置（{}）へのシークに
失敗しました。", path, start))?;
            buffer.resize(len, 0);
            file.read_exact(&mut buffer)
                .with_context(|| format!("ファイル '{}' の指定範囲（{} バイト）の読み
取りに失敗しました。", path, len))?;
        }
    } else {
        file.read_to_end(&mut buffer)
            .with_context(|| format!("ファイル '{}' の全内容の読み取りに失敗しまし
た。", path))?;
    }

    let (content, encoding) = decode_to_utf8(&buffer);
    Ok((content, encoding))
}
```

src/tools/get_outline.rs

```
use crate::tools::safe_read::safe_read;
use crate::parser::get_language;
use tree_sitter::{Parser, Node};
use anyhow::{Result, anyhow, Context};
use std::path::Path;
use serde::Serialize;

#[derive(Serialize)]
struct OutlineItem {
    line: u32,
    kind: String,
    name: String,
}

pub fn get_outline(path: &str) -> Result<String> {
    let (content, _) = safe_read(path, None, None)
        .with_context(|| format!("アウトライン取得のためのファイル読み取り（'{}'）に失
敗しました。", path))?;
```

```
    let extension = Path::new(path)
        .extension()
        .and_then(|s| s.to_str())
        .ok_or_else(|| anyhow!("ファイル '{}' に拡張子がありません。パーサーを決定するために拡張子が必要です。", path))?;

    let language = get_language(extension)
        .ok_or_else(|| anyhow!("拡張子 '.{}' はサポートされていません（ファイル: '{}'）。現在サポートされているのは .rs, .py, .cs です。", extension, path))?;

    let mut parser = Parser::new();
    parser.set_language(language)
        .with_context(|| format!("言語 '{}' 用のパーサーのセットアップに失敗しました。", extension))?;

    let tree = parser.parse(&content, None)
        .ok_or_else(|| anyhow!("ファイル '{}' の解析（パース）に失敗しました。ファイルが壊れているか、対応していない形式の可能性があります。", path))?;

    let mut outline = Vec::new();
    traverse(tree.root_node(), &content, &mut outline);

    serde_json::to_string(&outline)
        .with_context(|| format!("ファイル '{}' のアウトライン結果（JSON）の生成に失敗しました。", path))
}

fn traverse(node: Node, source: &str, outline: &mut Vec<OutlineItem>) {
    let kind = node.kind();

    let is_definition = match kind {
        // Rust
        "function_item" | "struct_item" | "enum_item" | "trait_item" | "mod_item" | "type_item" | "impl_item" => true,
        // Python
        "function_definition" | "class_definition" => true,
        // C#
        "class_declaration" | "method_declaration" | "struct_declaration" | "interface_declaration" | "enum_declaration" | "namespace_declaration" => true,
        _ => false,
    };

    if is_definition {
        let name_node = match kind {
            "function_item" | "struct_item" | "enum_item" | "trait_item" | "mod_item" | "type_item" => node.child_by_field_name("name"),
            "function_definition" | "class_definition" => node.child_by_field_name("name"),
            "method_declaration" | "class_declaration" | "struct_declaration" | "interface_declaration" | "enum_declaration" | "namespace_declaration" => node.child_by_field_name("name"),
            _ => None,
        };
```

```
        let name = if let Some(n) = name_node {
            n.utf8_text(source.as_bytes()).unwrap_or("unknown").to_string()
        } else if kind == "impl_item" {
            if let Some(type_node) = node.child_by_field_name("type") {
type_node.utf8_text(source.as_bytes()).unwrap_or("unknown").to_string()
            } else {
                "impl".to_string()
            }
        } else {
            "unknown".to_string()
        };

        let line = node.start_position().row as u32 + 1;
        outline.push(OutlineItem {
            line,
            kind: kind.to_string(),
            name,
        });
    }

    let mut cursor = node.walk();
    for child in node.children(&mut cursor) {
        traverse(child, source, outline);
    }
}
```

src/tools/inspect_file.rs

```
use std::fs;
use std::time::SystemTime;
use chrono::{DateTime, Utc};
use anyhow::{Result, Context};
use serde::Serialize;
use crate::encoding::decode_to_utf8;
use super::validate_and_canonicalize;

#[derive(Serialize)]
pub struct InspectResult {
    pub size: u64,
    pub created: String,
    pub modified: String,
    pub total_lines: usize,
    pub estimated_encoding: String,
    pub search_results: Vec<SearchResult>,
}

#[derive(Serialize)]
pub struct SearchResult {
    pub line_number: usize,
    pub context: Vec<String>,
```

```
}

pub fn inspect_file(path: &str, search_query: Option<String>) -> Result<String> {
    let path_buf = validate_and_canonicalize(path)?;
    let metadata = fs::metadata(&path_buf)
        .with_context(|| format!("ファイル '{}' のメタデータ（作成日時やサイズなど）を
取得できませんでした。", path))?;
    let size = metadata.len();

    let created = metadata.created().unwrap_or(SystemTime::UNIX_EPOCH);
    let modified = metadata.modified().unwrap_or(SystemTime::UNIX_EPOCH);

    let created_dt: DateTime<Utc> = created.into();
    let modified_dt: DateTime<Utc> = modified.into();

    let bytes = fs::read(&path_buf)
        .with_context(|| format!("ファイル '{}' の内容を読み取ることができませんでし
た。", path))?;
    let (content, encoding) = decode_to_utf8(&bytes);

    let lines: Vec<&str> = content.lines().collect();
    let total_lines = lines.len();

    let mut search_results = Vec::new();
    if let Some(query) = search_query {
        let query_lower = query.to_lowercase();
        for (i, line) in lines.iter().enumerate() {
            if line.to_lowercase().contains(&query_lower) {
                let start = if i >= 2 { i - 2 } else { 0 };
                let end = std::cmp::min(i + 3, total_lines);
                let context = lines[start..end].iter().map(|s|
s.to_string()).collect();

                search_results.push(SearchResult {
                    line_number: i + 1,
                    context,
                });

                // Limit search results to avoid huge output
                if search_results.len() >= 10 {
                    break;
                }
            }
        }
    }

    let result = InspectResult {
        size,
        created: created_dt.to_rfc3339(),
        modified: modified_dt.to_rfc3339(),
        total_lines,
        estimated_encoding: encoding.to_string(),
        search_results,
    };
```

```
    serde_json::to_string_pretty(&result)
        .with_context(|| format!("ファイル '{}' の解析結果をJSON形式に変換できませんでした。", path))
}
```

src/tools/read_hex.rs

```
use std::fs::File;
use std::io::{Read, Seek, SeekFrom};
use anyhow::{Result, Context};
use super::validate_and_canonicalize;

pub fn read_hex(path: &str, offset: Option<u64>, length: Option<usize>) -> Result<String> {
    let path_buf = validate_and_canonicalize(path)?;
    let mut file = File::open(&path_buf)
        .with_context(|| format!("バイナリ読み取りのためにファイル '{}' を開くことができませんでした。", path))?;
    let start_offset = offset.unwrap_or(0);
    let read_len = length.unwrap_or(256);

    file.seek(SeekFrom::Start(start_offset))
        .with_context(|| format!("ファイル '{}' のオフセット {} へのシークに失敗しました。", path, start_offset))?;

    let mut buffer = vec![0u8; read_len];
    let bytes_read = file.read(&mut buffer)
        .with_context(|| format!("ファイル '{}' のオフセット {} からのデータ読み取りに失敗しました。", path, start_offset))?;
    buffer.truncate(bytes_read);

    let mut output = String::new();
    for (i, chunk) in buffer.chunks(16).enumerate() {
        let current_offset = start_offset + (i * 16) as u64;

        // Address
        output.push_str(&format!("{:08x}: ", current_offset));

        // Hex values
        for j in 0..16 {
            if j < chunk.len() {
                output.push_str(&format!("{:02x} ", chunk[j]));
            } else {
                output.push_str("   ");
            }
            if j == 7 {
                output.push(' ');
            }
        }

        output.push_str(" ");
```

```
        // ASCII values
        for &b in chunk {
            if b >= 32 && b <= 126 {
                output.push(b as char);
            } else {
                output.push('.');
            }
        }

        output.push('\n');
    }

    Ok(output)
}

#[cfg(test)]
mod tests {
    use super::*;
    use std::io::Write;
    use std::fs;

    #[test]
    fn test_read_hex_logic() -> Result<()> {
        let test_file = "test_hex.tmp";
        let data = b"Hello, World!\x01\x02\x03\x04";
        let mut file = fs::File::create(test_file)?;
        file.write_all(data)?;

        let result = read_hex(test_file, Some(0), Some(17))?;
        println!("Hex output:\n{}", result);

        // クリーンアップ
        let _ = fs::remove_file(test_file);

        // 検証
        assert!(result.contains("00000000:"));
        // Check for specific hex parts
        assert!(result.contains("48 65 6c 6c 6f"));
        assert!(result.contains("57  6f 72 6c 64 21")); // Note the double space
after 57 (index 7)
        assert!(result.contains("Hello, World!"));
        Ok(())
    }
}
```

その他

Cargo.toml

```
[package]
name = "nen-mcp-server"
version = "0.1.0"
edition = "2021"

[dependencies]
serde = { version = "1.0", features = ["derive"] }
serde_json = "1.0"
encoding_rs = "0.8"
chardetng = "0.1"
tree-sitter = "0.20"
tree-sitter-python = "0.20"
tree-sitter-rust = "0.20"
tree-sitter-c-sharp = "0.20"
chrono = "0.4"
anyhow = "1.0"
windows-sys = { version = "0.52", features = ["Win32_System_Console",
"Win32_Foundation"] }
clap = { version = "4.5", features = ["derive"] }
dunce = "1.0"
```

README.md

```
# NEN (Non-English Normalization) MCP Server

マルチバイト文字（日本語など）を含むファイルの解析・読み込みを強力にサポートする MCP サーバーです。Windows 環境でのパス問題や文字化けを完全に解決します。

- [日本語](#日本語)
- [ENGLISH](#english)

---

## 日本語

### 概要
NEN MCP Server は、Windows 環境における PowerShell の文字化けや、Shift-JIS などの多様なエンコーディングが混在するプロジェクトでの file 操作を最適化するために設計されました。特に Windows 特有の UNC パス（`\\?\` 接頭辞）によるエラーを回避し、AI エージェントが非英語圏のコードベースを正確に理解するための「目」となります。

### 主な機能
- **Windows パス問題の解決**: `dunce` を採用し、Windows の拡張パス接頭辞に起因するファイルアクセスエラーを回避します。
- **自動エンコーディング検知**: UTF-8, Shift-JIS, EUC-JP などの文字コードを自動判別し、UTF-8 に正規化して読み込みます。
- **CLI/MCP ハイブリッドモード**: サーバーとしてだけでなく、コマンドラインツールとしても動作し、直接ファイルの検証が可能です。
- **アウトライン解析**: Tree-sitter を使用し、関数の定義などを構造的に抽出します。
```

インストール方法
```bash
gemini extensions install https://github.com/DovahkiinYuzuko/nen-mcp-server
```

CLI モードの使い方
```bash
# ファイルの安全な読み込み
./bin/nen-mcp-server.exe safe-read "path/to/ファイル.txt"

# MCPサーバーとして明示的に起動
./bin/nen-mcp-server.exe --mcp
```

ENGLISH

Overview
NEN (Non-English Normalization) MCP Server is designed to optimize file operations in environments with multi-byte characters (such as Japanese) and varied encodings. It effectively resolves Windows-specific path issues and terminal encoding corruption.

Key Features
- **UNC Path Fix**: Avoids `\\?\` prefix issues on Windows using `dunce` for reliable file access.
- **Automatic Encoding Detection**: Detects and normalizes UTF-8, Shift-JIS, and EUC-JP into standard UTF-8.
- **Hybrid CLI/MCP Mode**: Works both as a standardized MCP server and a standalone CLI tool.
- **Structural Analysis**: Extracts code structure (functions, classes) using Tree-sitter.

CLI Usage
```bash
# Standalone execution
./bin/nen-mcp-server.exe safe-read "path/to/file.txt"

# Force MCP mode
./bin/nen-mcp-server.exe --mcp
```

LICENSE

```
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.

---
# Third-Party Licenses
This project is built using the following open-source libraries, which are
licensed under the MIT License (or dual-licensed under MIT/Apache 2.0). We
gratefully acknowledge their contributions:
- anyhow (MIT / Apache-2.0)
- chardetng (MIT / Apache-2.0)
- chrono (MIT / Apache-2.0)
- encoding_rs (MIT / Apache-2.0)
- serde / serde_json (MIT / Apache-2.0)
- tree-sitter & its parsers (MIT)
- windows-sys (MIT / Apache-2.0)
- clap (MIT / Apache-2.0)
- dunce (MIT / Apache-2.0 / CC0-1.0)
---
```

.gitignore

```
# --- Build Artifacts ---
nen-mcp-server/target/

# --- Internal Development Tools ---
.qodo/
conductor/
docs/superpowers/plans/
docs/superpowers/specs/
nen-mcp-server/課題.md

# --- Temporary Test Files ---
test_outline.py
test_output_utf8.json
test_sjis.txt
Test.cs

# --- User Local Config ---
nen-mcp-server/mcp_config_example.json
```

```
# --- OS / IDE ---
.vscode/
.DS_Store
Thumbs.db
```

gemini-extension.json

```
{
  "name": "non-english-normalization-mcp",
  "version": "0.1.0",
  "description": "[ENG] A robust MCP server that completely resolves character encoding issues (such as Shift-JIS corruption) in Windows CLI environments, enabling AI agents to accurately read and parse non-English documents. [JPN] Windows CLI環境における文字化け（Shift-JIS等）を完全に解決し、AIエージェントが非英語圏のドキュメントを正確に読み取り、解析できるようにする堅牢なMCPサーバーです。",
  "mcpServers": {
    "nen-server": {
      "command": "${extensionPath}/bin/nen-mcp-server.exe",
      "args": []
    }
  },
  "contextFileName": "GEMINI.md"
}
```

nen-mcp-server/README.md

```
# Non-EnglishNormalization (NEN) MCP Server

Windows環境における文字化けを解消し、Tree-sitterによる構造抽出機能を提供する高精度なMCPサーバーです。

- [日本語](#日本語)
- [English](#english)

---

## 日本語

### 概要
NEN MCP Serverは、Windows環境で発生しやすいShift-JIS等の文字コードによる文字化け問題を解決し、ソースコードや文書の構造を正確に把握するためのツール群を提供します。
`chardetng`による高精度なエンコーディング自動検出と、`tree-sitter`によるシンタックス解析を組み合わせることで、AIエージェントの理解力を向上させます。

### 提供ツール

#### 1. `safe_read`
ファイルをバイナリで読み込み、エンコーディングを自動判別してUTF-8としてデコードします。
- `path` (必須): ファイルパス
```

- `range` (任意): 読み込み開始・終了バイト位置 `[start, end]`
- `tail` (任意): 末尾から読み込むバイト数

2. `get_outline`
Tree-sitterを使用して、ファイル内の関数やクラスの定義を構造的に抽出します。
- `path` (必須): ファイルパス
- 対応言語: Rust (`.rs`), Python (`.py`), C# (`.cs`)

3. `inspect_file`
ファイルのメタデータ（サイズ、作成日時、更新日時）の取得と、キーワード検索を同時に行います。
- `path` (必須): ファイルパス
- `search_query` (任意): 検索キーワード。一致した行とその前後2行をコンテキストとして返します。

4. `read_hex`
ファイルをヘキサダンプ形式で表示します。文字コード判別が困難なバイナリファイルや、破損したファイルの調査に最適です。
- `path` (必須): ファイルパス
- `offset` (任意): 開始オフセット
- `length` (任意): 読み込みバイト数（デフォルト 256）

セットアップ

ビルド
```powershell
cd nen-mcp-server
cargo build --release
```

設定
Claude DesktopなどのMCPクライアントで本サーバーを使用するには、設定ファイル（例: `%APPDATA%\Claude\claude_desktop_config.json`）に以下のように記述します。

```json
{
  "mcpServers": {
    "nen-server": {
      "command": "C:\\path\\to\\your\\nen-mcp-server\\target\\release\\nen-mcp-server.exe",
      "args": []
    }
  }
}
```

開発ルール
- すべての機能はUTF-8で正規化。
- パス検証ロジックを共通化し、Windows特有のUNCパス問題にも対応。
- Tree-sitterによる多言語サポート。

English

```
### Overview
NEN (Non-English Normalization) MCP Server is designed to resolve encoding issues
(such as Shift-JIS corruption) in Windows environments and provide structured code
analysis using Tree-sitter.
By combining high-precision encoding detection via `chardetng` with syntax parsing
via `tree-sitter`, it enhances the analytical capabilities of AI agents.

### Provided Tools

#### 1. `safe_read`
Reads a file in binary mode, automatically detects the encoding, and decodes it as
UTF-8.
- `path` (required): File path
- `range` (optional): Start and end byte positions `[start, end]`
- `tail` (optional): Number of bytes to read from the end

#### 2. `get_outline`
Uses Tree-sitter to extract function and class definitions from a file.
- `path` (required): File path
- Supported Languages: Rust (`.rs`), Python (`.py`), C# (`.cs`)

#### 3. `inspect_file`
Retrieves file metadata (size, creation time, modification time) and performs
keyword search simultaneously.
- `path` (required): File path
- `search_query` (optional): Search keyword. Returns the matching line and its
surrounding context.

#### 4. `read_hex`
Displays file content in a hex dump format. Ideal for inspecting binary files or
corrupted documents.
- `path` (required): File path
- `offset` (optional): Starting offset
- `length` (optional): Number of bytes to read (default: 256)

### Setup

#### Build
```powershell
cd nen-mcp-server
cargo build --release
```

Configuration
To use this server with an MCP client like Claude Desktop, add the following to
your config file:

```json
{
  "mcpServers": {
    "nen-server": {
      "command": "C:\\path\\to\\your\\nen-mcp-server\\target\\release\\nen-mcp-
server.exe",
```
```

```
      "args": []
    }
  }
}
```

Development Guidelines
- All outputs are normalized to UTF-8.
- Unified path validation logic, compatible with Windows UNC paths.
- Multi-language support via Tree-sitter.

GEMINI.md

```
# Non-EnglishNormalization (NEN) Extension Context

## Overview
This extension provides the `nen-server` MCP, which is designed to handle file
reading and structural parsing in environments where character encoding issues
(e.g., Shift-JIS on Windows) frequently occur.

## Usage Guidelines
When assisting the user with reading local files, analyzing source code structure,
or investigating binary files in this workspace, you MUST prioritize the tools
provided by `nen-server` over standard shell commands (`cat`, `type`, etc.) or
generic file reading tools.

### Available Tools:
1. **`safe_read`**:
   - **When to use**: Whenever you need to read the contents of a text file,
source code, or documentation.
   - **Why**: It automatically detects the character encoding (e.g., Shift-JIS,
UTF-8) and normalizes the output, preventing garbled text (mojibake) in your
context window.
   - **Note**: Supports `range` (line/byte specification) and `tail` for reading
large files efficiently.

2. **`get_outline`**:
   - **When to use**: When you need to understand the architecture, classes, or
functions within a source code file without reading its entire content.
   - **Why**: Uses Tree-sitter to parse the AST and return a concise JSON
structure of the file, saving tokens and improving comprehension.

3. **`inspect_file`**:
   - **When to use**: When you need metadata (size, lines, estimated encoding) or
need to search for a specific keyword within a file and view its surrounding
context.

4. **`read_hex`**:
   - **When to use**: When you need to inspect binary files, executables, or when
`safe_read` fails to decode a file properly. Returns a traditional hex dump.
```

```
## Strict Rules
- Do NOT use `cat` or `Get-Content` to read files if `safe_read` is applicable.
- If a file appears to contain garbled text when read with a standard tool,
immediately switch to using `safe_read`.
```